Optimus-Cost-Agent: Strategic Roadmap

Phase 2 & Phase 3 Distributed Architectures (v10)

Architected by: Vibhanshu Agarwal

1. Executive Summary

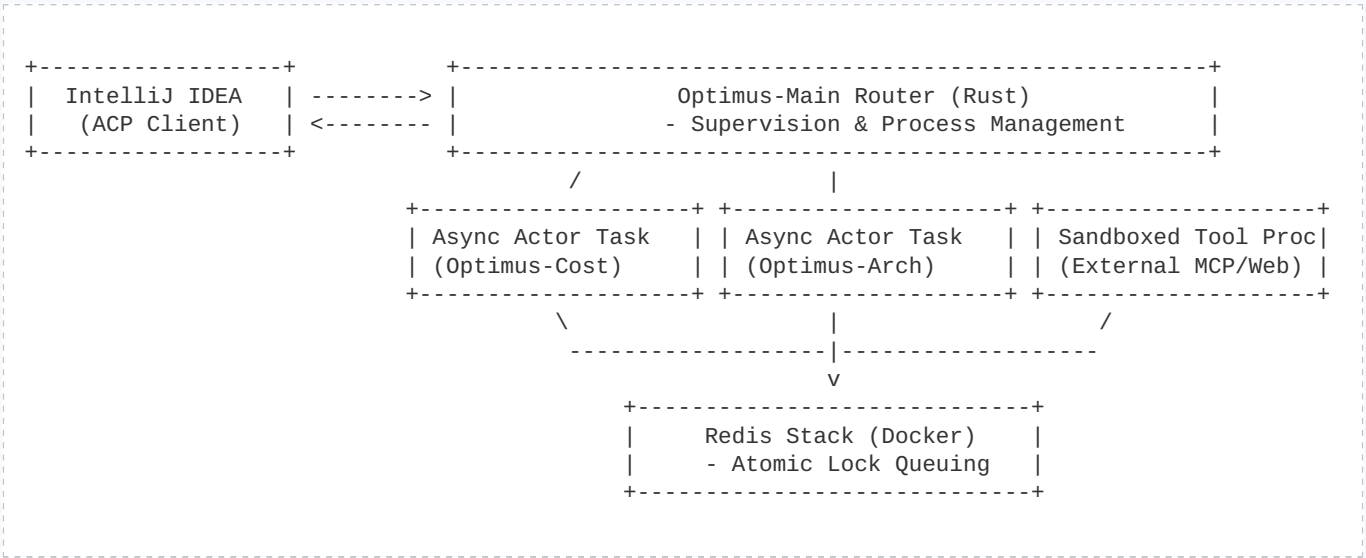
While Phase 1 of the Optimus-Cost-Agent establishes local request-level cost attribution and automated harness engineering, the subsequent roadmap targets native platform isolation and team federation. Phase 3 evolves the architecture into a fully governed **Agentic Mesh**, utilizing the Model Context Protocol (MCP) as the enterprise-wide connective tissue.

2. Phase 2: Rust Local Multi-Agent Swarm

Phase 2 decouples the runtime away from Python execution boundaries, moving core operations to a native Rust background daemon built using the Actor model (`tokio`). This shift emphasizes runtime safety, predictable idle state characteristics, and lifecycle isolation over raw memory savings:

- **Actor Supervision Loop:** Core agent routing, verification, and validation states run as lightweight asynchronous tasks coordinated by an explicit actor supervision layout to handle worker unexpected closures gracefully.
- **OS Process Isolation:** In contrast to internal actors, external tools, web scrapers, and Model Context Protocol (MCP) servers run outside the daemon boundary within sandboxed, low-privilege host operating system processes to completely shield active file hierarchies.
- **Stable Idle Footprint:** Eliminates runtime garbage collection halts, guaranteeing millisecond response hooks whenever an IDE protocol notification triggers.

Phase 2 High-Level Design (HLD)



3. Phase 3: Distributed Enterprise Agentic Mesh

Phase 3 transitions the platform from a local productivity utility to an interconnected **Agentic Mesh**. By treating architectural governance as a distributed concern, it bridges scattered local desktop environments with a centralized cluster orchestrator.

A. Agentic Context & Enterprise Governance Plane

To operate within strict enterprise security regulations and enforce cross-team standards, the centralized cluster incorporates multi-tier defense and context negotiation nodes:

- **Cross-Swarm MCP Negotiation:** Distributed Optimus daemons utilize MCP not just for local tool access, but to securely negotiate architectural state, dependencies, and API contracts across different business units in real-time. While MCP carries context and tool access, this enterprise negotiation layer strictly enforces auth, policy boundaries, schema versioning, and tenant isolation.
- **Hybrid Secrets Redaction:** Payloads hitting corporate networks undergo deterministic regex template scanning, high-entropy token analysis, and validation checking against locally configured secret registries or key allowlists to isolate proprietary patterns.
- **Telemetry Privacy Boundaries:** Fine-grained developer workspace opt-in constraints ensure metadata transfers are exclusively mapped to high-level usage attribution records, completely dropping underlying code strings.
- **Multi-Tenant Workspace Partitioning:** Enforces workspace data isolation boundaries across the shared execution pool, mapping roles and permissions cleanly.

Phase 3 High-Level Design (HLD)